



Hugging Face

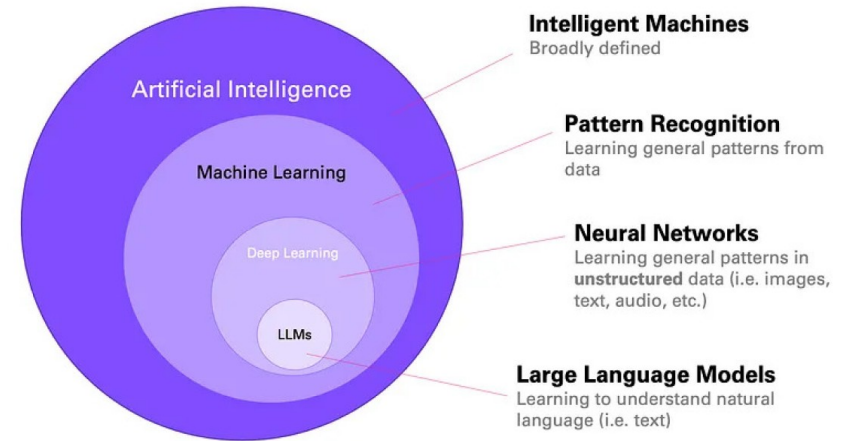
NLP und LLMs

NLP (Natural Language Processing):

- Verarbeitung menschlicher Sprache
- Sentiment-Analyse, Named Entity Recognition und maschinelle Übersetzung
- Ziel: Verstehen, Interpretieren und Generieren von Sprache

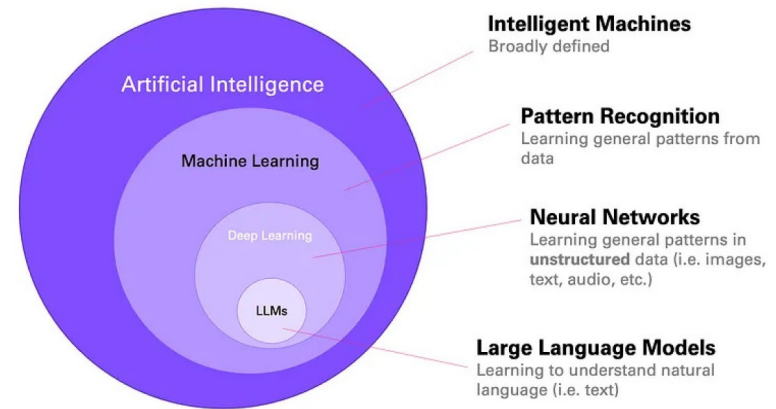
LLMs (Large Language Models):

- Spezielle Untergruppe innerhalb von NLP
- Basieren auf tiefen neuronalen Netzen mit Milliarden von Parametern und werden mit enormen Mengen an Textdaten trainiert.
- Können viele Sprachaufgaben ohne spezielles Fine-Tuning ausführen
- Beispiele: Llama-, GPT- oder Claude-Modelle



Large Language Models (LLMs)

- **Skalierung:** Enthalten Milliarden Parameter
- **Allgemeine Fähigkeiten:** Können verschiedene Aufgaben ohne spezifisches Training erledigen
- **In-Context Learning:** Lernen aus Beispielen, die direkt im Prompt gegeben werden
- **Neue Eigenschaften:** Mit wachsender Modellgröße treten Fähigkeiten auf, die nicht explizit programmiert oder erwartet waren

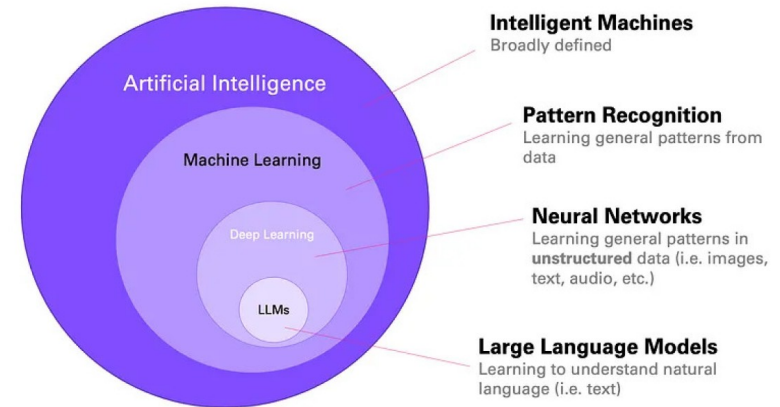


Anstatt für jede NLP-Aufgabe ein spezialisiertes Modell zu bauen, nutzt man nun ein einziges LLM, das durch Prompts oder Fine-Tuning auf unterschiedlichste Aufgaben angepasst werden kann.

Large Language Models (LLMs)

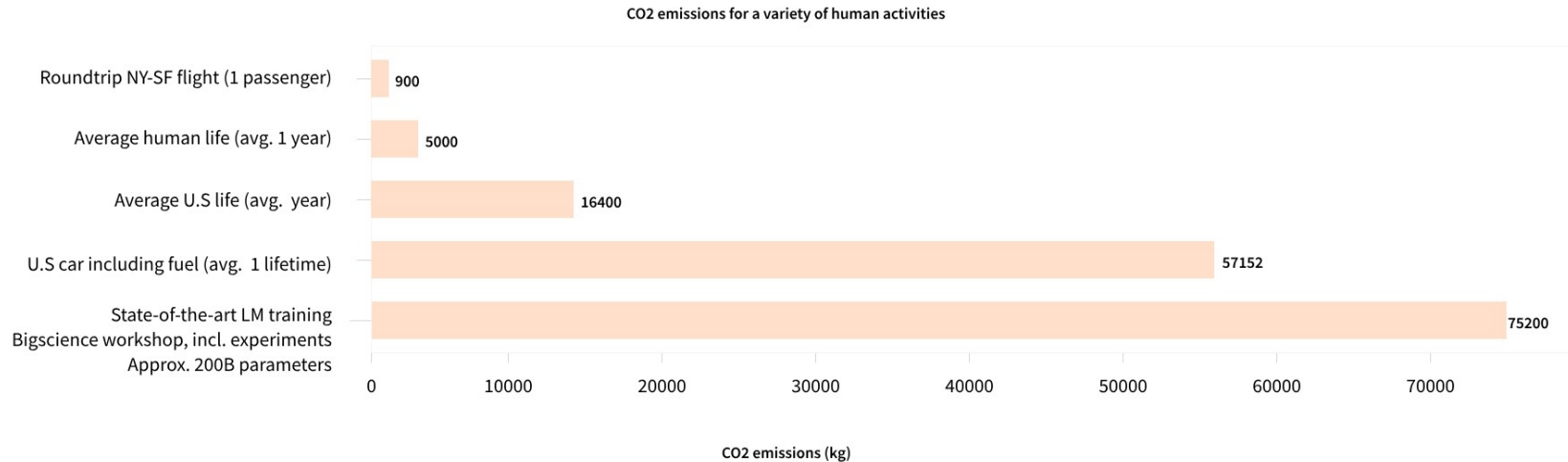
Einschränkungen

- **Halluzinationen:** Können falsche Informationen überzeugend erzeugen
- **Kein echtes Verständnis:** Arbeiten rein auf Basis statistischer Muster, ohne tiefes Weltverständnis
- **Bias:** Spiegeln Verzerrungen aus Trainingsdaten oder Eingaben wider
- **Kontextlänge:** Begrenzte Kontextfenster, die sich jedoch stetig verbessern
- **Rechenressourcen:** Benötigen enorme Mengen an Rechenleistung



Anstatt für jede NLP-Aufgabe ein spezialisiertes Modell zu bauen, nutzt man nun ein einziges LLM, das durch Prompts oder Fine-Tuning auf unterschiedlichste Aufgaben angepasst werden kann.

Large Language Models (LLMs)

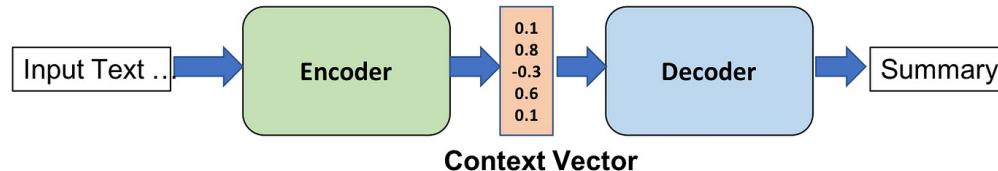


Transformers

- Transformer sind auf großen Textmengen trainiert werden.
- Transferlernen (Fine-Tuning) passt das Modell für spezifische Aufgaben an.

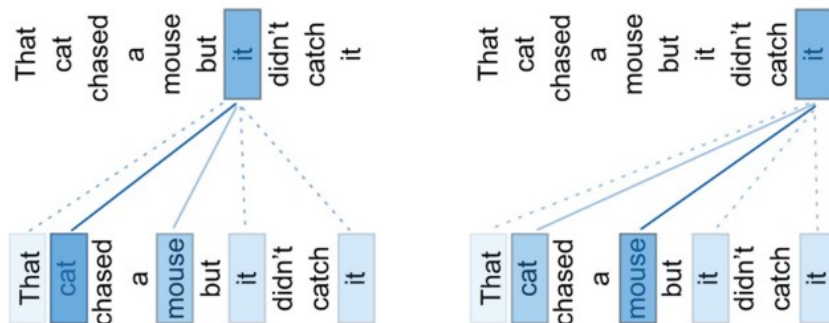
Encoder-Decoder-Architektur

- Encoder: Wandelt Eingaben (Text, Bild, Code) in verarbeitbare Vektor-Repräsentationen um
- Decoder: Auf Anfrage erzeugt die KI neue Inhalte, indem sie relevante Kontexte und Abhängigkeiten aus der Repräsentation nutzt (z. B. Chatantworten, Zusammenfassungen, Übersetzungen)
- Encoder-only, Decoder-only und Encoder-Decoder Modelle für verschiedene Aufgaben.



Transformers

- Self-Attention ist ein Mechanismus, mit dem ein Transformer erkennt, welche Wörter in einem Satz für jedes andere Wort besonders wichtig sind.
- Das Modell kann dadurch Zusammenhänge im gesamten Satz berücksichtigen und nicht nur die direkte Nachbarschaft eines Wortes.
- Dadurch versteht der Transformer komplexe Bedeutungen und kann Texte präziser verarbeiten oder übersetzen.



Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

<https://arxiv.org/abs/1706.03762>

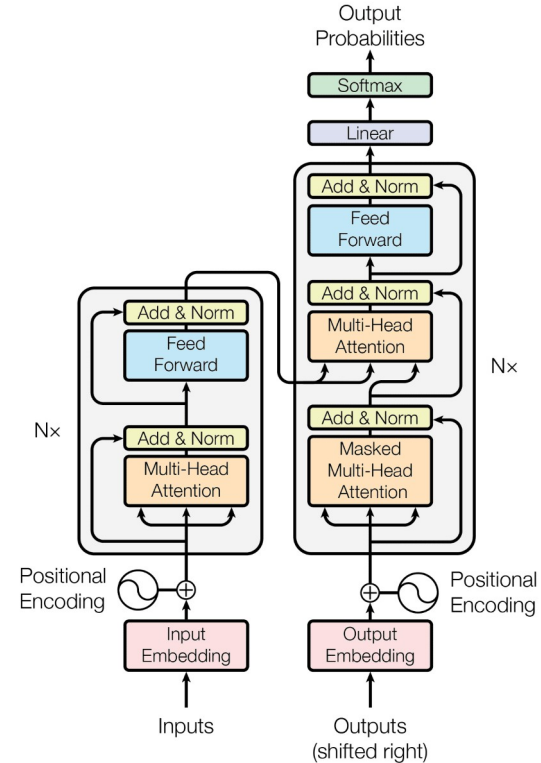
Transformers

Encoder

- Multi-Head Self-Attention: jedes Token kann mit jedem anderen Token interagieren. Für jedes Token wird berechnet, wie stark es auf andere Tokens „achten“ soll.
- Feedforward-Netzwerk (erkennt komplexere Zusammenhänge)
- Ziel: Input in eine kontextualisierte Repräsentation zu überführen - jedes Token enthält Informationen über den gesamten Input

Decoder

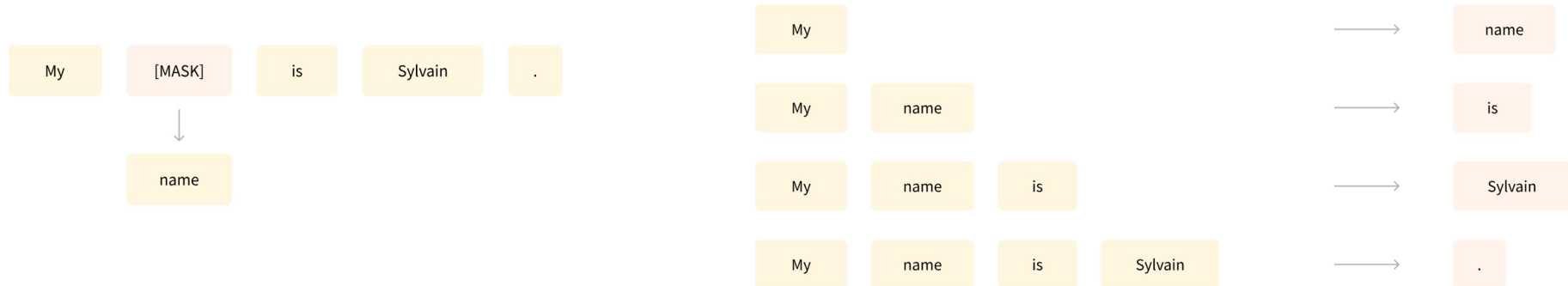
- Masked Self-Attention: verhindert, dass das Modell in die Zukunft schaut; für jedes neue Wort wird berechnet, wie wichtig die bereits generierten Wörter sind.
- Cross-Attention (Nutzt die vom Encoder gewonnenen Informationen)
- Feedforward-Netzwerk
- Ziel: Schrittweise die Output zu generieren - jedes neue Token wird auf Basis der bisherigen Ausgabe und der Encoder-Informationen erzeugt.



Understand something, then produce something else.

Transformers

- Sprachmodelle lernen, die Wahrscheinlichkeit von Wörtern im Kontext vorherzusagen → grundlegendes Sprachverständnis
- **MLM (Masked Language Modeling)**: Wörter maskieren → Modell rekonstruiert sie mit Blick auf gesamten Kontext (z. B. BERT)
- **CLM (Causal Language Modeling)**: Nächstes Wort vorhersagen → nur vorherige Wörter nutzbar (z. B. GPT)



Transformers

- GPT steht für **Generative Pretrained Transformer (decoder-only)**.

1. Vortrainieren (Pretraining):

Das Modell wird mit riesigen Mengen an Textdaten aus dem Internet „gefüttert“.

Ziel: Das nächste Wort in einem Satz vorhersagen → so lernt es Grammatik, Fakten und Zusammenhänge.

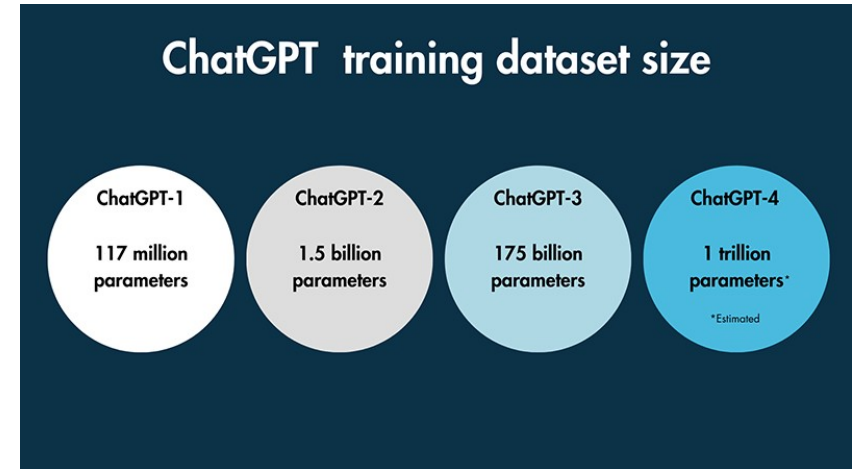
2. Feinabstimmung (Fine-Tuning):

Nach dem Vortraining wird das Modell mit gezielten Daten weiter trainiert.

Oft unter menschlicher Aufsicht („Reinforcement Learning with Human Feedback“ – RLHF).

3. Anwendung:

Das trainierte Modell kann dann Texte verstehen und selbst erzeugen – z. B. als Chatbot, Textgenerator oder Assistent.



Architektur

1. Tokenisierung

Der Eingabetext wird in kleine Einheiten (Tokens) zerlegt, z.B. Wörter oder Teilwörter.

2. Einbettung

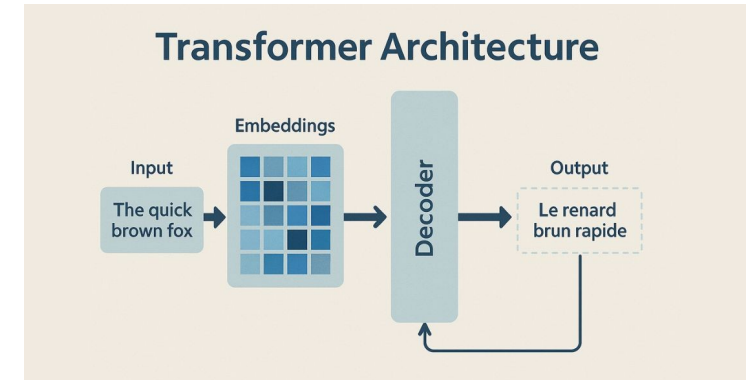
Jedes Token wird in einen Vektor im hochdimensionalen Raum umgewandelt. Dadurch kann das Modell semantische Ähnlichkeiten und Unterschiede zwischen Wörtern erfassen.

3. Kontextverarbeitung

Die Transformer-Schichten analysieren die Beziehungen aller Tokens zueinander mithilfe des Self-Attention-Mechanismus. So kann das Modell auch weit entfernte Zusammenhänge im Text erkennen.

4. Vorhersage & Dekodierung

Das Modell berechnet die Wahrscheinlichkeit für jedes mögliche nächste Token und wählt das wahrscheinlichste aus. Verschiedene Strategien (z.B. Sampling, Beam Search) steuern, wie kreativ oder exakt die Ausgabe ist.



Tokenisierungsmethoden

Subword-Tokenisierung

- Zerlegt Wörter in kleinere Einheiten (Subwörter oder Morpheme) und bildet so einen Mittelweg zwischen Wort- und Zeichen-Tokenisierung.
- Byte-Pair Encoding (BPE), WordPiece, SentencePiece.
- Geht mit seltenen und unbekannten Wörtern sowie morphologisch reichen Sprachen um; **Standard in modernen Sprachmodellen (z. B. BERT, GPT).**
- Unzufriedenheit" → ["Un", "zufrieden", "heit"] oder ["Un", "zu", "frieden", "heit"]
- Komplexer, manchmal unnatürliche Wortaufteilungen.

Zeichen-Tokenisierung

- "hilfreich" → ["h", "i", "l", "f", "r", "e", "i", "c", "h"]
- Hilfreich bei Sprachen ohne Wortgrenzen (z. B. Chinesisch), Rechtschreibkorrektur oder detaillierter Textanalyse.
- Bedeutungsverlust auf Wortebene, sehr lange Sequenzen.

Large Language Models (LLMs), such as GPT-3 and GPT-4, utilize a process called tokenization. Tokenization involves breaking down text into smaller units, known as tokens, which the model can process and understand. These tokens can range from individual characters to entire words or even larger chunks, depending on the model. For GPT-3 and GPT-4, a Byte Pair Encoding (BPE) tokenizer is used. BPE is a subword tokenization technique that allows the model to dynamically build a vocabulary during training, efficiently representing common words and word fragments. Although the core tokenization process remains similar across different versions of these models, the specific implementation can vary based on the model's architecture and training objectives.

Hugging Face

- Plattform für den Zugriff auf und das Experimentieren mit Open-Source Large Language Models (LLMs).
- Flexibilität: Modelle können für spezifische Domänen optimiert werden.
- **Hugging Face Hub:** Eine Plattform mit vielen vortrainierten Modellen.
- **Transformers-Bibliothek:** Einsatz und das Fine-Tuning von LLMs.
- **HuggingChat:** Eine Open-Source-Alternative zu ChatGPT mit Llama 2.

Hugging Face

